ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ

МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ

БИЕ ДААЛТЫН ТАЙЛАН

ҮЙЛДЛИЙН СИСТЕМ (F.CSM302)

2026-2027 оны хичээлийн жилийн намар

Сэдэв: Kernel data structures
Бие даалтын зорилго: Сурах бичигт заасан kernel data structure-ийн хэрэглээг хэрэгжүүлж, элемент нэмэх, устгах, хайх, үйлдлийн дараах бүтцийг графаар харуулна.

Хичээл заасан багш: Г. Ганбат /F.CS10/
Бие даалтын ажил гүйцэтгэсэн: Э.Тэргэл: B210930801
Б.Булгантамир: B241940123
Т.Төгөлдөр: B242270050
И.Зэлмэ: B231940158
О.Амина: B241910800
М.Наранбат: B242270801

Гарчиг

ХУРААНГУЙ 2
1. УДИРТГАЛ 3
1.1 Зорилго 3
1.2 Ач холбогдол 3
1.3 Хамрах хүрээ 3
2. СУРАХ БИЧГИЙН ШААРДЛАГА 3
2.1 Эх сурвалж 3
2.2 Шаардлага ба хэрэгжилтийн харьцуулалт 3
2.3 Хуваарилалт 4
3. ОНОЛЫН ҮНДЭС 4
3.1 Intrusive circular doubly linked list 4
3.1.1 Бүтэц 4
3.1.2 HEAD sentinel 4
3.1.3 Logical ID 4
3.2 Алгоритмын өртөг 4
3.3 Collatz дараалал 5
4. СИСТЕМИЙН ЗОХИОМЖ 5
4.1 Компонент ба өгөгдлийн урсгал 5
4.2 Интерфейсийн гэрээ 5
4.3 Зэрэгцээ хандалт ба эрх 5
5. ХЭРЭГЖИЛТ 6
5.1 colors.c 6
5.2 Collatz.c 6
6. ТУРШИЛТ БА UI ДҮРСЛЭЛ 6
6.1 UI-ийн бүтэц 6
6.2 Operation history ба export 6
6.3 Practice simulation ба Live mode 6
7. ТУРШИЛТЫН АРГА 6
7.1 Туршилтын түвшин 6
7.2 Туршилтын тохиолдлууд 7
7.3 Номын параметрийн жишээ 7
7.4 Kernel acceptance 7
8. БОДИТ ҮР ДҮН БА НОТОЛГОО 7
8.1 Локал автомат шалгалт 7
8.2 Kernel compilation 8
8.3 Screenshot-ийн шаардлага 8
9. ХЭЛЭЛЦҮҮЛЭГ 8
9.1 Үр дүнгийн тайлбар 8
9.2 Хязгаарлалт 8
10. ДҮГНЭЛТ 9
10.1 ..... Хэрэгжсэн зүйл ба нотолсон үр дүн 9
10.2 ..... GitHub ажиллагаа ба AI хэрэглээ 9
АШИГЛАСАН МАТЕРИАЛ 9

ХУРААНГУЙ

Энэ ажлаар Linux цөмд өгөгдлийг динамикаар хадгалах, холбоосоор дамжуулан боловсруулах, ашигласан санах ойг буцаан чөлөөлөх арга барилыг судаллаа. Part I-д RGB өнгө агуулсан дөрвөн элементтэй intrusive circular doubly linked list-ийг, Part II-д start параметрээс эхлэх Collatz дарааллыг kernel module хэлбэрээр тус тус хэрэгжүүлсэн.

Python HTTP adapter нь /proc интерфейсээс бүтцийн төлөвийг уншиж, HTML/SVG UI дээр next болон prev холбоосыг дүрслэн харуулдаг. Элемент нэмэх, ID-аар устгах, RGB-ээр хайх, жагсаалтыг хоослох, анхны төлөвт нь буцаах командуудыг цөм рүү дамжуулна.

Локал орчинд Python болон HTTP тестүүд, мөн kernel module-тэй хамт ашигладаг C арифметикийн тестийг ажиллуулсан. Баталгаажсан болон баталгаажаагүй зүйлсийг 8-р хэсэгт тусад нь тодорхой заасан. Practice mode-ийн үр дүнг kernel runtime-ийн нотолгоо гэж үзээгүй.

1. УДИРТГАЛ

1.1 Зорилго

Төслийн зорилго нь kernel дахь санах ой хуваарилалт, intrusive list, жагсаалтаар гүйх үйлдэл, аюулгүй устгал, module-ийн амьдралын мөчлөг, мөн user болон kernel орчны хилийг бодит кодоор тайлбарлах явдал юм.

1.2 Ач холбогдол

Үйлдлийн систем нь процесс, төхөөрөмж, нөөцийн бүртгэлийг динамик өгөгдлийн бүтцээр зохион байгуулдаг. Linked list нь урт нь урьдчилан тогтоогдоогүй бөгөөд node-ийн хаяг мэдэгдэж байвал холбоосыг тогтмол хугацаанд өөрчлөх боломжтой. Харин хайлт хийхдээ бүх элементийг дараллаар нь шалгах шаардлагатай болдог.

1.3 Хамрах хүрээ

Төсөл нь Part I-ийн RGB өнгөтэй intrusive circular doubly linked list kernel module, Part II-ийн start параметрээс үүсэх Collatz дарааллын тусдаа kernel module, /proc интерфейс, локал Python HTTP adapter, graph UI болон автомат тестийг хамарна. Production authentication, remote hosting, олон browser-ийн тусгаарлагдсан session төлөв нь төслийн хүрээнд ороогүй.

2. СУРАХ БИЧГИЙН ШААРДЛАГА

2.1 Эх сурвалж

Operating System Concepts, 10th edition, Chapter 3, Programming Project 4 (хэвлэмэл P-19–P-22). Номын Part II нь заавал гүйцэтгэх, тусдаа Collatz module юм.

2.2 Шаардлага ба хэрэгжилтийн харьцуулалт

Хүснэгт 1. Шаардлага ба хэрэгжилт

Шаардлага	**Хэрэгжилт**
Part I: init үед дөрвөн өнгө үүсгэх	colors_init → reset_list
Жагсаалтаар гүйж kernel log-д хэвлэх	list_for_each_entry + pr_info
Exit үед устгаж, санах ойг чөлөөлөх	list_for_each_entry_safe + list_del + kfree
Part II: start параметр	collatz.ko, module_param(start, ullong, 0)
Collatz жагсаалт ба traversal	sequence list, value, id, next/prev
Add/delete/search хийсний дараах граф	/proc/kds_colors + interactive SVG
Өөр оролт, хязгаарын болон алдаатай утга	RGB, ID, capacity, overflow-ийн шалгалт
Дахин ажиллуулах заавар	README, Manual, tests, Makefile

2.3 Хуваарилалт

Сургалтын хуваарийн дагуу II-4-4 нь 4-р сэдэв бөгөөд UI нь Kernel structure graph байна. Шүүмжлэх баг нь IV-2-7 бөгөөд тэдний 7-р сэдэв нь Fork-Join Sorting Application юм.

3. ОНОЛЫН ҮНДЭС

3.1 Intrusive circular doubly linked list

3.1.1 Бүтэц

struct color дотор RGB өгөгдлөөс гадна struct list_head байрлана. list_head нь next болон prev холбоосыг агуулна. Өгөгдлийн node бүр холбоосоо өөртөө агуулдаг учир intrusive гэж нэрлэдэг.

3.1.2 HEAD sentinel

HEAD нь өгөгдлийн элемент биш, sentinel node юм. Жагсаалт хоосон үед HEAD.next болон HEAD.prev нь өөрийг нь заана. Элемент байвал HEAD.next нь эхний, HEAD.prev нь сүүлийн node-ийг зааж, сүүлийн node-ийн next болон эхний node-ийн prev нь HEAD руу буцна.

3.1.3 Logical ID

UI нь kernel хаягийг задруулдаггүй. ID 0 нь HEAD-ийг илэрхийлнэ. Бусад node-ийн ID-г list_head холбоосоос node_id() функцээр гаргана. Иймээс next ба prev утга нь frontend дээрх жагсаалтын индекс биш, бодит холбоосыг харуулна.

3.2 Алгоритмын өртөг

Хүснэгт 2. Үйлдлүүдийн хугацаа болон санах ойн өртөг

Үйлдэл	Хугацаа	Тайлбар
Tail insert	O(1)	list_add_tail
RGB search	O(n)	эхний тохирох элемент олдтол шалгана
Delete by ID	O(n)	хайсны дараа unlink хийх нь O(1)
Traversal / graph snapshot	O(n)	элемент бүрийг уншина
Clear / unload	O(n)	node бүрийг kfree хийнэ
Санах ой	O(n)	node бүр өгөгдөл болон хоёр холбоос агуулна

3.3 Collatz дараалал

Эхлэх n нь эерэг бүхэл тоо байна. n тэгш бол n/2, сондгой бол 3n+1 гэж тооцоолж, n=1 болмогц зогсооно. Бүх эерэг тооны хувьд дараалал төгсдөг гэдэг нь батлагдсан теорем биш тул жагсаалтыг 4096 node-оор хязгаарласан. Мөн unsigned 64-bit overflow гарахаас өмнө n > (ULL_MAX-1)/3 нөхцөлийг шалгана.

4. СИСТЕМИЙН ЗОХИОМЖ

4.1 Компонент ба өгөгдлийн урсгал

Команд нь Browser-loopback Python HTTP adapter-/proc/kds_colors-colors.ko-kernel list гэсэн дарааллаар дамжина. Команд биелсний дараа шинэ snapshot уншиж, граф, хүснэгт болон тохирсон ID-г шинэчилнэ.

Collatz module нь load хийх үед жагсаалтаа үүсгээд, дараа нь өөрчлөгдөхгүй (immutable) snapshot өгнө. Параметрийг солих бол terminal дээр unload, load хийх шаардлагатай; UI дээрх Refresh товч шинэ snapshot-ийг уншина. Web server нь shell команд ажиллуулахгүй.

4.2 Интерфейсийн гэрээ

Хүснэгт 3. /proc/kds_colors командууд

Команд	Утга / үр дүн
ADD r g b	0-255 утгатай RGB нэмж, шинэ logical ID олгоно
SEARCH r g b	Эхний тохирох ID-г буцаана, олдохгүй бол 0
DELETE id	ID-аар устгана, олдохгүй бол ENOENT
CLEAR	Бүх node-ийг чөлөөлнө
RESET	Анхны дөрвөн node-ийг сэргээнэ (ID 1-4)

/proc-ийн JSON нь kind, head.next, head.prev, nodes болон colors-ийн found утгыг агуулна. Collatz-ийн value-г JSON string хэлбэрээр өгдөг тул JavaScript-ийн 53-bit бүхэл тооны хязгаараас болж утга алдагдахгүй.

4.3 Зэрэгцээ хандалт ба эрх

colors-ийн унших, бичих үйлдлүүд нэг mutex ашиглана. RESET нь шинэ жагсаалтыг бүрэн allocate хийсний дараа хуучныг солих тул allocation алдаатай бол хуучин төлөв хэвээр үлдэнэ. Unload үед proc_remove эхлээд интерфейсийг салгаж, дараа нь санах ойг цэвэрлэнэ.

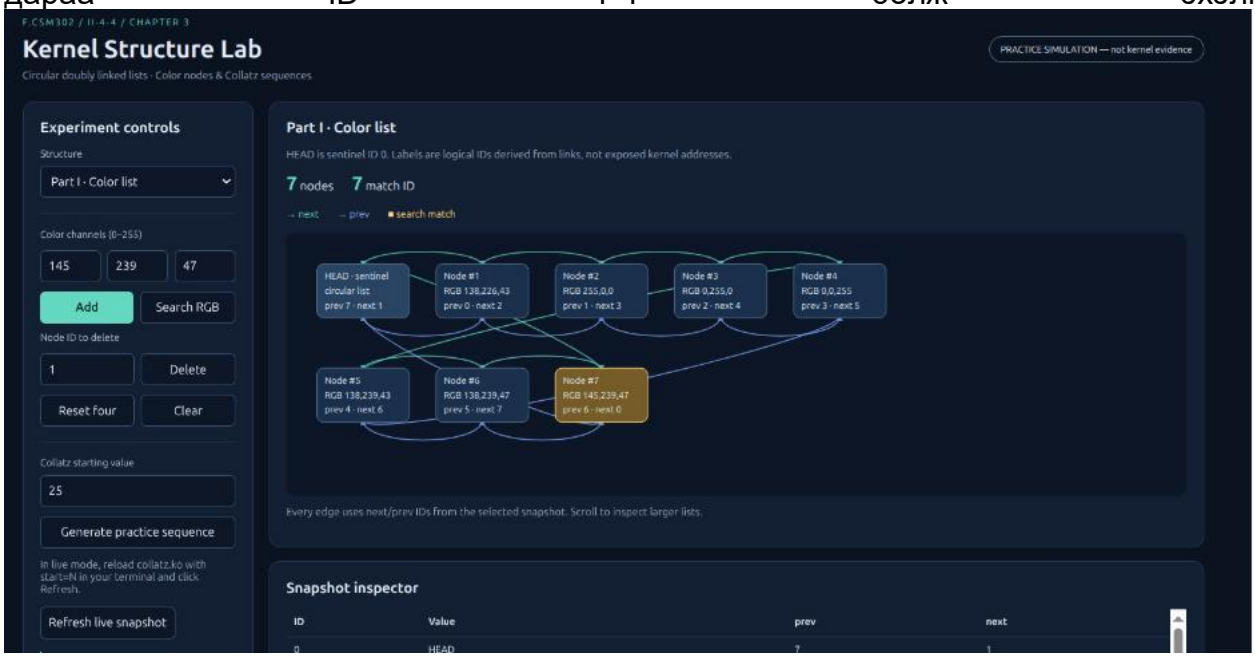
Proc файлууд зөвхөн root эрхтэй. Бичих үйлдэл CAP_SYS_ADMIN эрхийг шалгана. Server нь 127.0.0.1 дээр ажиллаж, POST хүсэлт бүр дээр Host, Origin болон session token-ийг шалгана. Энэ нь сургалтын VM-д зориулсан локал хэрэгсэл бөгөөд production service биш.

5. ХЭРЭГЖИЛТ

5.1 colors.c

colors.c нь list.h, kmalloc/kfree, mutex, proc_ops, seq_file ашиглана. RGB parser kstrtoint, ID parser kstrtouint ашигладаг бөгөөд хэт том тоо, дутуу эсвэл илүү token, embedded NUL тохиолдолд алдаа буцаана. Capacity нь 128 node. Давхардсан RGB утга зөвшөөрөгдөх ба node бүр өөр ID-тай байна.

reset_list нь эхлээд түр жагсаалт үүсгэнэ. Дөрвөн allocation бүгд амжилттай болсны дараа list_splice_tail_init-ээр үндсэн жагсаалтад шилжүүлнэ. clear_list нь safe iterator ашиглан устгана. CLEAR хийхэд ID дахин 1-ээс эхлэхгүй; RESET хийсний дараа ID 1-4 болж эхэлнэ.



5.2 Collatz.c

Collatz.c нь start параметрийг авч linked list үүсгэнэ. Allocation, overflow, capacity эсвэл proc_create-д алдаа гарвал үүсгэсэн бүх node-ийг чөлөөлж, module-ийг load хийхээс татгалзана. Төгсгөлийн 1 утгыг мөн жагсаалтад хадгална.

F.CSM302 / II-4-4 / CHAPTER 3

Kernel Structure Lab

Circular doubly linked lists · Color nodes & Collatz sequences

PRACTICE SIMULATION — not kernel evidence

Experiment controls

Structure

Part II · Collatz list

Color channels (0–255)

145 | 239 | 47

Add | Search RGB

Node ID to delete

1 | Delete

Reset four | Clear

Collatz starting value

27

Generate practice sequence

In live mode, reload collatz.ko with start=N in your terminal and click Refresh.

Refresh live snapshot

Completed COLLATZ

Part II · Collatz sequence

HEAD is sentinel ID 0. Labels are logical IDs derived from links, not exposed kernel addresses.

112 nodes — match ID

→ next → prev ■ search match



Every edge uses next/prev IDs from the selected snapshot. Scroll to inspect larger lists.

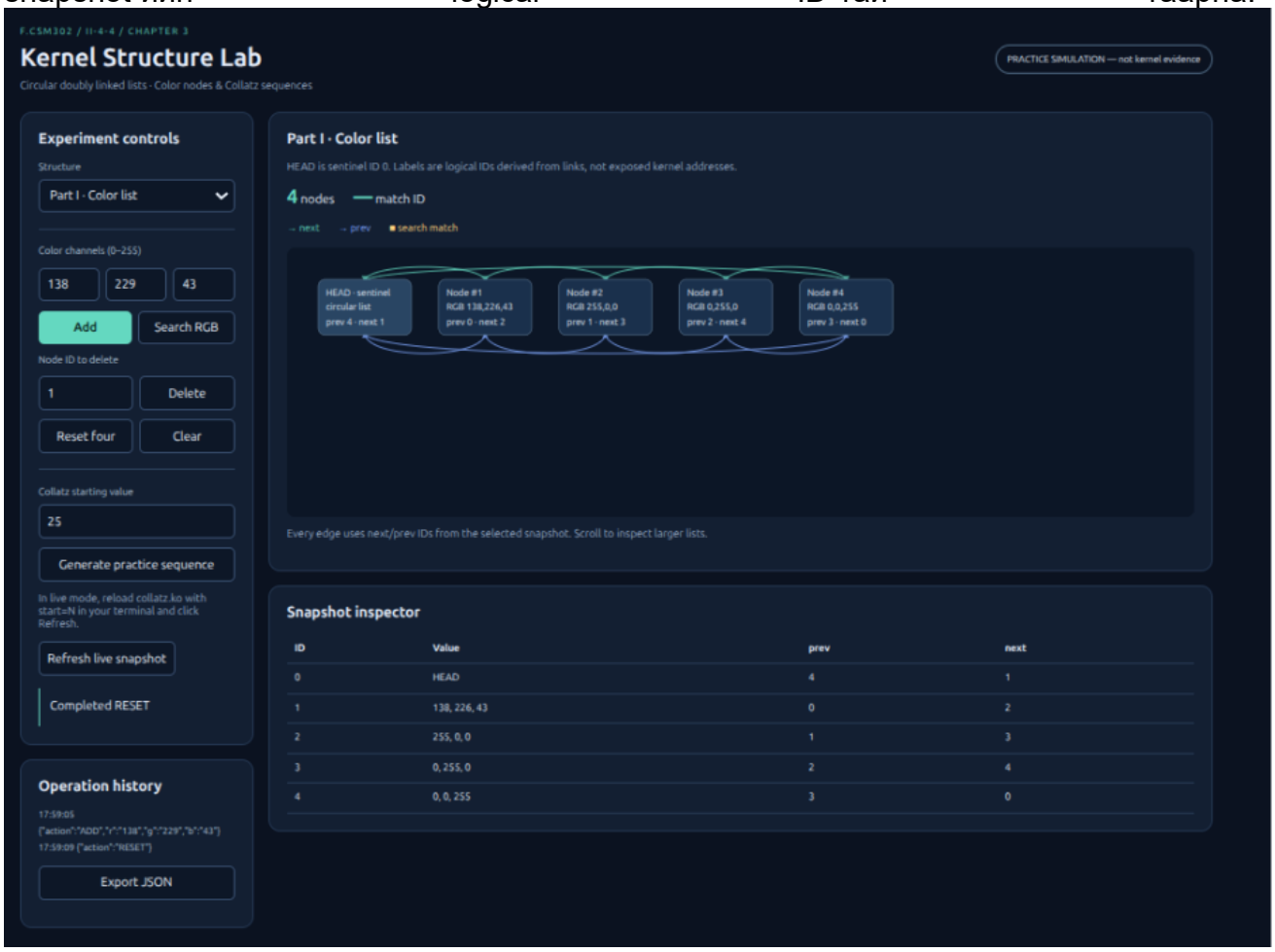
Snapshot inspector

106	10	105	107
107	5	106	108
108	16	107	109
109	8	108	110
110	4	109	111
111	2	110	112
112	1	111	0

6. ТУРШИЛТ БА UI ДҮРСЛЭЛ

6.1 UI-ийн бүтэц

Зүүн талд RGB, ID, start оролт болон командын товч бүхий control panel байна. Баруун талд HEAD ба node-ууд, ногоон next болон цэнхэр prev холбоос, хайлтаар олдсон node-ийн шар хүрээ, snapshot хүснэгт харагдана. Графын node, edge нь snapshot-ийн logical ID-тай таарна.



6.2 Operation history ба export

Operation history нь тухайн browser session дэх амжилттай командууд, timestamp болон snapshot-ийг хадгална. Export JSON нь одоогийн төлөв болон history-г татаж авна. Энэ нь kernel log-ийг орлохгүй бөгөөд dmesg-ийн нотолгоог тусад нь хадгална.

6.3 Practice simulation ба Live mode

Practice simulation нь Windows, Linux, macOS дээр kernel module-гүйгээр ажиллана. Live mode нь module байхгүй бол алдааг харуулах бөгөөд practice mode руу нууцаар шилжихгүй. Practice горимын screenshot-ийг live kernel screenshot гэж тайлагнаж болохгүй.

7. ТУРШИЛТЫН АРГА

7.1 Туршилтын түвшин

Локал тестүүд гурван түвшинтэй: model болон validation, HTTP contract, kernel module-д ашиглагддаг нийтлэг C арифметик. Kernel acceptance script нь тусдаа Ubuntu VM дээр бодит module-ийг insmod, rmmod хийж /proc-ийг шалгах зорилготой.

7.2 Туршилтын тохиолдлууд

Хүснэгт 4. Туршилтын тохиолдол ба хүлээгдэх үр дүн

Тест	**Оролт**	**Хүлээгдэх үр дүн**
Initial list	load/reset	4 node, хоёр талын холбоост дугуй бүтэц
CRUD	ADD 10 20 30, SEARCH, DELETE ID 5	олдоно; дараа нь устна
Boundary deletion	1, 3, 4, 2 ID-г дарааллаар	HEAD өөрийгөө заасан хоосон жагсаалт
Missing	DELETE 999	алдаа; төлөв өөрчлөгдөхгүй
Invalid RGB	-1, 256, текст, 2^32	татгалзана
Duplicates	Red нэмж хайх	эхний тохирох ID нь 2
Capacity	128 node-ийн дараах 129 дэх	ENOSPC; жагсаалт хэвээр
Collatz	1 / 15 / 25 / 27	1 / 18 / 24 / 112 node
Overflow	ULL_MAX	load болон үүсгэлтээс татгалзана
HTTP controls	token-гүй / гадны Origin	403

7.3 Номын параметрийн жишээ

start=15 үед: 15 → 46 → 23 → 70 → 35 → 106 → 53 → 160 → 80 → 40 → 20 → 10 → 5 → 16 → 8 → 4 → 2 → 1.

7.4 Kernel acceptance

Kernel acceptance нь edge бүрийн next.prev болон prev.next invariant-ийг шалгана. Unload хийсний дараа /proc entry алга болсныг шалгана. dmesg нь load үеийн traversal болон cleanup-ийн тоог баталгаажуулах баримт болно.

Cleanup log дангаараа memory leak байхгүйг бүрэн нотлохгүй тул kmemleak зэрэг kernel instrumentation-ийг нэмж ашиглаж болно.

8. БОДИТ ҮР ДҮН БА НОТОЛГОО

8.1 Локал автомат шалгалт

2026-11-01-нд local automation-ийг ажиллуулсан. Raw log: docs/evidence/local-tests.txt. ASAN_OPTIONS=detect_leaks=0 тохиргоог энэ container-ийн LeakSanitizer process inspection хязгаарлалтаас шалтгаалан хийсэн.

Хүснэгт 5. Бодит шалгалтын төлөв

Шалгалт	**Бодит төлөв**
Python model + HTTP	22 тест PASS
Shared C arithmetic	GCC, ASan/UBSan PASS
JavaScript parsing	Syntax PASS
Kernel compilation	PASS: Ubuntu 6.8.0-138 headers, W=1
Kernel insmod/rmmod	Энэ орчинд ажиллуулаагүй
Live UI screenshot	Баг Ubuntu VM дээр авна
Browser visual acceptance	Browser binary байхгүй тул хүлээгдэж байна

8.2 Kernel compilation

Хоёр module Ubuntu 6.8.0-138-generic headers, GCC 13.3.0 дээр W=1 тохиргоотой compile болон link хийхэд амжилттай болсон. Raw log: docs/evidence/kernel-build.txt. Compiler package-ийн revision зөрүүний warning гарсан ба vmlinux байхгүйн улмаас BTF алгассан; source код дээр warning гараагүй. Host kernel нь 6.18.57 тул энэ .ko-г load хийгээгүй.

8.3 Screenshot-ийн шаардлага

Энд kernel runtime-ийн бодит screenshot зохиож оруулаагүй. Эцсийн хувилбарт дараах screenshot-ийг нэмэх шаардлагатай: colors-ийн анхны төлөв, хайлтын тодруулга (search highlight), хоосон жагсаалт, Collatz 15, dmesg-ийн load/unload. Screenshot бүр дээр mode, оролт, хүлээгдсэн болон бодит үр дүн, огноо, kernel version-ийг тэмдэглэнэ.

9. ХЭЛЭЛЦҮҮЛЭГ

9.1 Үр дүнгийн тайлбар

Kernel implementation болон practice model ижил үр дүн өгсөн ч энэ нь kernel-д онцлог бүх алдааг үгүйсгэхгүй. Allocation алдаа, kernel locking, module-ийг устгах, ABI тохироо зэргийг live орчинд заавал шалгах шаардлагатай хэвээр. Kernel headers дээр compile амжилттай болсон нь ажиллаж буй kernel дээр load амжилттай болохыг батлахгүй.

9.2 Хязгаарлалт

UI нь kernel pointer хаягийг биш, логик topology-г харуулдаг. Их хэмжээний Collatz жагсаалтын үед SVG өндөр болох тул scroll болон snapshot хүснэгтээр шалгана. Production authentication, remote hosting, олон browser-ийн тусгаарлагдсан session төлөв нь төслийн хүрээнд ороогүй.

10. ДҮГНЭЛТ

10.1 Хэрэгжсэн зүйл ба нотолсон үр дүн

Номын хоёр хэсэгт зориулсан kernel module, add/delete/search/reset интерфейс, graph UI, автоматжуулалт болон дахин ажиллуулах баримт бичгийг боловсруулсан. Локал тестүүд нь input validation, model invariant, HTTP interface болон Collatz арифметикийн сонгосон нөхцлүүдийг баталгаажуулсан.

Эцсийн хувилбар болгохын өмнө Ubuntu VM дээр kernel build, load, run, unload, live UI болон бодит screenshot-ийг баталгаажуулах шаардлагатай. Тайлангийн баталгаажуулалтын төлвийг зөвхөн бодит үр дүнгээр шинэчилнэ.

10.2 GitHub ажиллагаа ба AI хэрэглээ

Энэ хувилбарыг upload хийхэд бэлэн багц хэлбэрээр бэлтгэсэн бөгөөд GitHub холболт push=false тул remote commit, tag үүсгээгүй.

ChatGPT-ийг код, тест, UI, тайлан, Manual боловсруулахад ашигласан. Python стандарт сан, Linux kernel API-г, PDF үүсгэхэд ReportLab-ыг ашигласан.

АШИГЛАСАН МАТЕРИАЛ

[1] A. Silberschatz, P. B. Galvin, G. Gagne. Operating System Concepts. 10th edition. Wiley, 2018. Chapter 3, Project 4, P-19-P-22; Section 1.9.

[2] Linux Kernel Documentation. Linked Lists in Linux. https://docs.kernel.org/core-api/list.html

[3] Linux Kernel Documentation. seq_file. https://docs.kernel.org/filesystems/seq_file.html

[4] Linux Kernel Documentation. Building External Modules. https://docs.kernel.org/kbuild/modules.html

[5] F.CSM302 Бие даалт 1-ийн удирдамж; сэдэв ба шүүмжийн хуваарилалт; Peer Review удирдамж (өгсөн PDF файлууд).